IES Luis Braille – Coslada

Departamento de Informática

Titulación:	DAM	**Curso:**	2.º
Módulo:	Acceso a Datos	**Año:**	2026–2027
Examen:	B – Árboles y JSON	**Fecha:**	_____
Duración:	_____	**Nota:**	

Nombre y apellidos:	
RA:	RA1 (evaluación global)

Módulo: Acceso a Datos · 2.º DAM

Modalidad: Práctica individual en ordenador

Material permitido: IDE y autocompletado. **No se permite consultar apuntes.**

Puntuación total: 10 puntos.

1. Contexto y recursos proporcionados

Vas a desarrollar una aplicación que procesa un inventario de árboles, consulta un árbol a partir de un identificador guardado en un fichero binario y genera nuevos ficheros de salida. Se evalúa globalmente el **RA1: gestión de información almacenada en ficheros**.

Se entrega un **proyecto Java preparado** que incluye:

- La clase `Arbol`, con sus getters, orden natural, `toJson()` y un `toString()` legible.
- El método `ParseadorArboles.parsear(String linea)`, que convierte un objeto JSON en un árbol. **No debes implementarlo ni modificarlo.**
- La clase `OperacionesArboles`, que resuelve la fusión de mapas, la incorporación sin duplicados y la ordenación para la salida.
- La petición de la ruta del directorio de salida y la coordinación de los ejercicios, ya implementadas.
- Tres ficheros de texto y un fichero binario en `datos/entrada`.

Archivo	Contenido
`arboles1.jsonl`	Un único árbol válido.
`arboles2.jsonl`	200 árboles válidos con IDs diferentes.
`arboles3.jsonl`	Árboles válidos, IDs repetidos y líneas incorrectas.
`id_arbol.bin`	Un ID guardado como entero binario de 32 bits.

Formato de texto: se utiliza **JSON Lines**, con extensión `.jsonl`: cada línea contiene un objeto JSON completo y compacto. No hay cabecera, array exterior ni comas entre líneas. Esto permite leer y tratar cada registro de forma independiente. Los ficheros se leen y escriben en **UTF-8**.

Cada árbol contiene exactamente `id`, `nombre`, `especie`, `edad` y `altura`. El ID y la edad son enteros;

la altura es un número en metros; el nombre y la especie son cadenas. El anexo incluye un ejemplo.

Comportamiento del parseador: devuelve un `Arbol`, lanza `FormatoJsonIncorrectoException` por sintaxis o propiedades incorrectas, lanza `TipoDatoIncorrectoException` por tipos incorrectos o devuelve `null` si hay valores de dominio inválidos. Los casos están documentados en el proyecto. No se evalúa el parseo JSON ni la lógica de negocio.

Convenciones del examen:

- Las colecciones de los ejercicios 3 y 5 se guardan en un `HashMap<Integer, Arbol>` cuya clave es `arbol.getId()` y cuyo valor es el árbol.
- Un árbol está repetido si su ID ya está presente. Se conserva la primera aparición; un registro posterior no debe reemplazarla.
- El orden natural es por especie, después nombre y después ID. Un `HashMap` no garantiza ese orden; usa la ayuda proporcionada al escribir el fichero final.
- Se completan exclusivamente los seis métodos `TODO` de `ExamenRA1B.java`. No se modifican los datos, el modelo, el parseador ni las clases auxiliares.
- Utiliza las rutas y los nombres proporcionados por las constantes del proyecto.

2. Ejercicios

Ejercicio 1. Preparación del directorio de salida – 1 punto

La aplicación ya recibe por consola la ruta del directorio de salida y la convierte en un `Path`.

Implementa la gestión de dicha ruta para que:

1. Compruebe si el directorio existe.
2. Si no existe, lo cree, incluidos los directorios intermedios necesarios.
3. Si ya existe y es un directorio, no lo modifique.
4. Muestre un mensaje indicando si se ha creado, si ya existía o si ha ocurrido un problema.
5. Si la ruta existe pero **no es un directorio**, comunique el error y no continúe escribiendo allí.

Devuelve `true` únicamente cuando el directorio de salida sea válido.

Ejercicio 2. Lectura de un árbol y escritura de texto – 1 punto

Lee `arboles1.jsonl` utilizando `ParseadorArboles.parsear(...)`.

Crea en el directorio de salida `arbol1.txt`, con **únicamente el nombre del árbol** seguido de un salto de línea. Obtén el nombre del objeto `Arbol`, no de la cadena JSON.

Gestiona recursos y errores de lectura y escritura. Devuelve el árbol válido leído para el ejercicio 6; si falla su escritura, conserva ese objeto. Si no se obtiene un árbol válido, devuelve `null`.

Ejercicio 3. Lectura de un mapa de árboles – 2 puntos

Lee `arboles2.jsonl` y guarda los árboles en un `HashMap<Integer, Arbol>`.

1. Utiliza una lectura apropiada para un fichero de texto con varias líneas.
2. Procesa cada línea con el parseador proporcionado.
3. Incorpora cada árbol con su ID como clave.
4. Gestiona los recursos y las excepciones.

5. Muestra por consola cuántos árboles se han incorporado al mapa.

Resultado esperado con el fichero facilitado: 200 árboles en el mapa.

Ejercicio 4. Lectura de un ID binario y consulta del árbol – 1 punto

El proyecto proporciona `id_arbol.bin`. Contiene **un entero binario de 32 bits**, con el formato que lee `DataInputStream.readInt()` (big-endian). No contiene caracteres de texto.

1. Abre el fichero y lee el número almacenado.
2. Busca ese ID como clave en el mapa obtenido en el ejercicio 3.
3. Si existe, imprime el árbol por pantalla utilizando su `toString()` proporcionado.
4. Si no existe, informa de que no hay un árbol con ese ID.
5. Gestiona el cierre del recurso y los errores de lectura, incluido un fichero incompleto, sin impedir que se ejecuten los ejercicios siguientes.

No debes pedir el ID por consola ni leer el fichero como texto. El archivo facilitado contiene el ID **1100**, presente en `arboles2.jsonl`.

Ejercicio 5. Fichero con errores y duplicados – 2 puntos

Lee `arboles3.jsonl` y construye un nuevo `HashMap<Integer, Arbol>`.

En él encontrarás árboles válidos, IDs repetidos **dentro del fichero 3**, objetos con propiedades ausentes o adicionales, JSON mal formado, campos de tipo incorrecto y valores que hacen que el parseador devuelva `null`.

1. Incorpora únicamente árboles válidos y no repetidos, usando el ID como clave.
2. Conserva la primera aparición de cada ID e ignora los duplicados **sin reemplazarla**.
3. Trata separadamente `FormatoJsonIncorrectoException`, `TipoDatoIncorrectoException` y el caso `null`.
4. Informa del número de línea (empezando en 1) y del motivo de cada descarte, incluidos los duplicados.
5. Continúa con las siguientes líneas tras errores recuperables de un registro.
6. Muestra al final cuántos árboles válidos se han incorporado.

No modifiques el parseador ni los ficheros para corregir los datos.

Ejercicio 6. Integración y generación del fichero final – 3 puntos

Trabaja con los mapas de los ejercicios 3 y 5 y con el árbol del ejercicio 2. La lógica de integración y ordenación ya está resuelta en `OperacionesArboles`.

Parte A. Primera generación

1. Obtén el mapa combinado con `OperacionesArboles.combinar(arboles2, arboles3)`. Esta ayuda conserva el árbol del fichero 2 cuando un ID está en ambos mapas.
2. Recorre `OperacionesArboles.ordenar(mapa)` para obtener los árboles en su orden natural.
3. Crea `arboles_ordenados.jsonl`. Escribe cada árbol en una línea mediante `arbol.toJson()` y añade un salto de línea.
4. **Usa** `StandardOpenOption.CREATE_NEW` para esta primera creación. Si el fichero ya existe, informa de ello y **no lo modifiques**.

Parte B. Actualización tras incorporar el árbol 1

5. **Solo si la Parte A ha creado el fichero y ha terminado su escritura correctamente**, llama a `OperacionesArboles.incorporarSiAusente(mapa, arbol1)`.
6. Recorre de nuevo `OperacionesArboles.ordenar(mapa)`.
7. Escribe **otra vez la colección completa** en el mismo fichero, reemplazando su contenido anterior, sin añadir una segunda copia al final.

Importante: `CREATE_NEW` se usa solo en la primera creación. La segunda escritura debe truncar el contenido anterior. Si el fichero existía antes de la Parte A, se omite también la Parte B y el fichero permanece intacto. Si falla la primera escritura, informa y no ejecutes la Parte B.

3. Criterios generales de corrección

Además del resultado, se valorará dentro de cada ejercicio:

- Uso adecuado de las API de ficheros y directorios de Java.
- Gestión correcta de recursos, usando `try-with-resources` cuando proceda.
- Tratamiento de errores sin silenciarlos ni interrumpir la lectura por errores recuperables de una línea.
- Distinción entre texto JSON y el entero almacenado en un fichero binario.
- Uso del `HashMap` con el ID como clave y conservación de la primera aparición.
- Cumplimiento de las condiciones de creación y actualización.
- Código claro y uso de las ayudas proporcionadas para centrarse en los ficheros.

Ejercicio	Puntuación
1. Directorio de salida	1
2. Fichero 1: texto	1
3. Fichero 2: HashMap	2
4. Lectura binaria y consulta por ID	1
5. Fichero 3: errores y duplicados	2
6. Integración y fichero final	3
Total de los ejercicios	**10**

Resultado de aprendizaje

RA1 (texto normativo): Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.